

Linked List Programs — Step-by-Step Solutions

Problem 1: Difference Between Minimum and Maximum in a Singly Linked List

Given list: $34 \rightarrow 77 \rightarrow 22 \rightarrow 55 \rightarrow 66 \rightarrow 11$

We need to traverse the singly linked list once, track the minimum and maximum values seen, and finally compute their difference.

Algorithm (Step by Step)

1. Create the singly linked list nodes and link them in the order $34 \rightarrow 77 \rightarrow 22 \rightarrow 55 \rightarrow 66 \rightarrow 11$.
2. Initialize two variables, min and max, with the data of the first (head) node.
3. Start a pointer 'temp' at the head node.
4. Traverse the list node by node using $\text{temp} = \text{temp} \rightarrow \text{next}$ until temp becomes NULL.
5. At each node, compare $\text{temp} \rightarrow \text{data}$ with min: if $\text{temp} \rightarrow \text{data} < \text{min}$, update $\text{min} = \text{temp} \rightarrow \text{data}$.
6. At each node, compare $\text{temp} \rightarrow \text{data}$ with max: if $\text{temp} \rightarrow \text{data} > \text{max}$, update $\text{max} = \text{temp} \rightarrow \text{data}$.
7. After the traversal ends, compute $\text{difference} = \text{max} - \text{min}$.
8. Print the minimum, maximum, and the difference.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

// Insert a new node at the end of the list
struct Node* insertEnd(struct Node *head, int value) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        return newNode;

    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    return head;
}

// Find difference between max and min values in the list
void findMinMaxDifference(struct Node *head) {
    if (head == NULL) {
        printf("List is empty\n");
    }
}
```

```

        return;
    }

    int min = head->data;
    int max = head->data;
    struct Node *temp = head;

    while (temp != NULL) {
        if (temp->data < min)
            min = temp->data;
        if (temp->data > max)
            max = temp->data;
        temp = temp->next;
    }

    printf("Minimum value = %d\n", min);
    printf("Maximum value = %d\n", max);
    printf("Difference (Max - Min) = %d\n", max - min);
}

int main() {
    struct Node *head = NULL;
    int values[] = {34, 77, 22, 55, 66, 11};
    int n = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < n; i++)
        head = insertEnd(head, values[i]);

    findMinMaxDifference(head);

    return 0;
}

```

Dry Run

Node visited	data	min (updated)	max (updated)
1	34	34	34
2	77	34	77
3	22	22	77
4	55	22	77
5	66	22	77
6	11	11	77

Output

```

Minimum value = 11
Maximum value = 77
Difference (Max - Min) = 66

```

Problem 2: Display Doubly Linked List in Forward & Reverse Order and Find the Average

A doubly linked list (DLL) node has three fields: prev, data, and next. We will build a DLL, print it from head to tail (forward) and from tail to head (reverse), and compute the average of the n elements.

Algorithm (Step by Step)

9. Create DLL nodes, each having prev, data, and next fields, and link them so next of one node points to the following node and prev points back to the previous node.
10. To display in forward order: start a pointer at head, print temp->data, move temp = temp->next, repeat until temp is NULL.
11. While moving forward, also keep a running sum (sum += temp->data) and a count 'n' (n++) so the average can be computed later.
12. To display in reverse order: move a pointer to the last node (tail) of the list — either by traversing to the end or maintaining a tail pointer directly.
13. From the tail, print temp->data, move temp = temp->prev, repeat until temp is NULL, which prints the list back to front.
14. Compute average = sum / n (use floating point division so the result is not truncated).
15. Print the forward list, the reverse list, and the average.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};

// Insert a new node at the end of the DLL, keep tail pointer updated
struct Node* insertEnd(struct Node *head, int value, struct Node **tail) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = NULL;

    if (head == NULL) {
        *tail = newNode;
        return newNode;
    }

    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
    *tail = newNode;
    return head;
}

// Display list forward, and simultaneously accumulate sum and count
void displayForward(struct Node *head, int *sum, int *count) {
    struct Node *temp = head;
    printf("Forward : ");
    *sum = 0;
    *count = 0;
    while (temp != NULL) {
        printf("%d ", temp->data);
        *sum += temp->data;
        (*count)++;
        temp = temp->next;
    }
}
```

```

    printf("\n");
}

// Display list in reverse, starting from the tail node
void displayReverse(struct Node *tail) {
    struct Node *temp = tail;
    printf("Reverse : ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    printf("\n");
}

int main() {
    struct Node *head = NULL, *tail = NULL;
    int values[] = {34, 77, 22, 55, 66, 11};
    int n = sizeof(values) / sizeof(values[0]);
    int sum, count;

    for (int i = 0; i < n; i++)
        head = insertEnd(head, values[i], &tail);

    displayForward(head, &sum, &count);
    displayReverse(tail);

    printf("Average of %d elements = %.2f\n", count, (float)sum / count);

    return 0;
}

```

Dry Run (using the same 6 values)

List used: 34 ↔ 77 ↔ 22 ↔ 55 ↔ 66 ↔ 11

Pass	Order of traversal	Result
Forward	head → next → next → ...	34 77 22 55 66 11
Reverse	tail → prev → prev → ...	11 66 55 22 77 34
Sum / Count	34+77+22+55+66+11 = 265, n = 6	sum = 265, n = 6

Output

```

Forward : 34 77 22 55 66 11
Reverse : 11 66 55 22 77 34
Average of 6 elements = 44.17

```

Note

The average is computed as sum / n using a float cast so decimals are preserved ($265 / 6 = 44.166\dots$, displayed as 44.17). If an integer average were required instead, the division would truncate to 44.